

PetSim
-hungerLevel : int -boredomLevel : int -name : string
+Talk() : void +FeedPet() : void +Play() : void -PetMood() : int -PassTime(in time : int = 1) : void +DisplayPetBehavior() : void +Menu() : void

The class also contains an overloaded constructor, set and get functions.

The PetSim puts the player in charge of his own virtual pet. The player is completely responsible for keeping the pet happy. He can feed and play with the pet to keep it in a good mood. He can also listen to the pet to learn how the pet is feeling, which can range from happy to mad. If you fail to feed or entertain your pet, it will have a mood change for the worse. But, with the proper care your pet can return to a sunny mood. NOTE: Improvise the game if necessary.

Constructor

The class must contain an overloaded constructor (no default constructor) to initialize data members. The constructor takes three arguments, hunger, boredom and name. The constructor also prints the initial statistics of the pet.

PetMood()

The pet's mood is directly based on its hunger and boredom levels. This function returns the sum of pet's hunger and boredom levels, a pet's mood gets worse as the number increases. This member function is private because it should only be invoked by another member function of the class.

Talk()

This function allows the pet to be able to tell the player how it's feeling. The pet's mood can be happy, okay, frustrated, or mad which is determined by the value returned by PetMood() function. The pet is happy if mood is less than 7, Okay if mood is 7 or higher, frustrated if the mood is between 16 and 21 and mad if greater than 21. Finally, Talk() calls PassTime() to simulate the passage of time.

FeedPet()

The FeedPet function allows the player to be able to feed the pet to reduce its hunger. FeedPet() displays an appropriate message indicating that the pet has been fed. This function reduces pet's hunger level by a certain amount. This amount can be passed as a parameter or as default parameter. The max amount of food that can be fed to the pet is value of 4. The pet's hunger level is kept in check and is not allowed to go below zero. Finally, PassTime() is called to simulate the passage of time.

Play()

The Play function allows the player to be able to reduce the pet's boredom. Play() displays an appropriate message indicating that the pet was allowed to play. The function reduces a pet's boredom level by a certain amount. This amount can be passed as a parameter or as default parameter. The max amount of fun a pet can have is value of 4. The pet's boredom level is kept in check and is not allowed to go below zero. Finally, PassTime() is called to simulate the passage of time.

PassTime()

PassTime() is a private member function that **increases** a pet's hunger and boredom levels by adding the specified amount of time to the member variables hungerLevel and boredomLevel. It is invoked at the end of each member function where the pet does something (eats, plays, or talks) to simulate the passage of time. You can pass the member function the amount of time that has passed; otherwise, time gets the default argument value of 1.

DisplayPetBehavior()

This function displays appropriate pet behavior based on the hunger level and boredom levels. This function must also display the actual hunger and boredom levels.

The hunger and boredom levels less than 3 indicate that the pet is neither hungry nor bored. Boredom levels between 3 and 11 indicate that the pet is a little bored, otherwise it is very bored. Hunger level between 3 and 11 indicate that the pet is moderately hungry. Hunger levels greater than 11 indicate that the pet is very hungry. This function is called by member functions Talk(), FeedPet() and Play().

Menu ()

It is basically be a game loop that asks the player whether he wants to listen to, feed, or play with the pet, or quit the game. Here's the pseudocode:

```
While the player doesn't want to quit the game
  Present a menu of choices to the player
  If the player wants to listen to the pet
    Make the pet talk
  If the player wants to feed the pet
    Feed the pet
  If the player wants to play with the pet
    Make the pet play
```

main()

In main() instantiates a new Pet object. You can supply specific values for Name, Hunger or Boredom or leave it at default values. When the data members start out at 0, the pet begins life happy and content.

Set and Get Functions

These functions must provide appropriate code. The set functions must also validate the hunger and boredom levels. The hunger and boredom levels cannot be less than 0 and cannot be greater than 100. Ensure that these functions are used wherever necessary and not the member variables directly.

Rubrics:

	Measurement of performance of aspect.
Aspect	Objectives Substantially Met
Correctness Max Points : 10.5	1. Overloaded constructor is used with specified parameters; the constructor correctly initializes the member variables and prints the initial pet's stats. (0.5 Points) 2. Set functions validate the hunger and boredom level and have appropriate code. (0.5 Points) 3. The get functions have appropriate code. (0.5 Points) 4. PetMood function has the specified code and this function is called using correct syntax. (0.5 Points) 5. Talk function contains appropriate control statements to perform the task and correctly displays the appropriate pet mood. (0.5 Points) 6. FeedPet reduces pets hunger levels as specified and displays appropriate message indicating that the pet has been fed. (0.5 Points) 7. Play reduces pets boredom levels as specified and displays appropriate message indicating the same. (0.5 Points) 8. PassTime has code that increments the hunger and boredom levels as specified and this function is called using correct syntax. (0.5 Points) 9. DisplayPetBehavior displays the actual hunger and boredom levels to the user and uses appropriate control statement(s) to perform the task. This function is invoked using correct syntax by the specified member functions. (0.5 Points) 10. Menu displays the specified options to the user and uses an anonymous enumerator and a switch..case along with any other appropriate control structure(s) to perform the task. This function must correctly invoke the appropriate functions based on the options user selects. (0.5 Points) 11. Main contains minimal code required to run the program. (0.5 Points) 12. Program uses the set and get functions to store or retrieve the hunger and boredom levels. 13. Program runs and completes all required tasks. (3 Point) 14. Program enhancements to make the game more interesting (up to 1Point) 15. Executes without errors (Logical). (1 Point)
User Interface Max Points : 2	1. Specification is followed correctly and includes all appropriate, descriptive and user friendly input prompts (1 Point) 2. Specification is followed correctly and output is user-friendly, and clearly describes what is expected from the user. (1 Point)

Documentation Max Points : 5.5	1. Program contains required project commenting header (for each file) with correct format and clearly describes the purpose of the project. (1.5 Point) 2. Program contains function commenting headers clearly describing the purpose of the function. (2.5 Point) 3. Program contains comments for the statements where ever necessary. (1 Points) 4. Self-commenting variable, method and class names are used. (0.5 Points) NOTE: Documentation must correctly outline inheritance, composition, member initializer list and calls for the base class constructor where ever required.
Construction Max Points : 2	1. OOPs concepts must be used ie., in this case the C++ classes must be used to complete the program. Procedural or any other programming is not allowed. 2. Tasks are modularized and well defined. 3. Each class should have a .h file for class declaration, and a .cpp file for the implementation. All control structures must use {} even if the control structure contains only one statement. (1 Point) 4. Source code is logically laid out, using current best practices and with proper indentation. 5. The students should use camel-casing notation only throughout the program. Students must use appropriate variables names [identifiers] and functions throughout the program. 6. Students must use data types efficiently. 7. Global variables are not used. The only global functions that is allowed is main(). (1 Point)